ViajaMejor — Presentación del Caso de Negocio

Documento para la Evaluación Parcial N°1 (EP01) · JVY0101 Java: Diseño y Construcción de Soluciones Nativas en Nube

---

1. Presentación de la empresa

ViajaMejor es una plataforma de reservas de viajes que integra hoteles, vuelos, arriendo de vehículos y paquetes turísticos. Permite buscar disponibilidad, reservar, pagar en línea y administrar la fidelización de los viajeros.

- **Rubro:** TravelTech / OTA (Agencia de viajes en línea)
- **Usuarios objetivo:** viajeros frecuentes, familias y empresas
- **Escala esperada:** picos de reservas en temporada alta y eventos masivos
- **Modelo de negocio:** comisión por reserva y planes de fidelización

2. Problema / necesidad de negocio

Los viajes requieren coordinar múltiples proveedores (aerolíneas, hoteles, arriendo de vehículos) con disponibilidad que cambia constantemente y precios dinámicos. Una falla en pagos no debe bloquear la búsqueda de disponibilidad, y las reservas deben persistir de forma confiable aunque un proveedor externo falle.

3. Requisitos funcionales (RF)

Código	Requisito	Dominio
RF-01	Registro y autenticación de viajeros	Usuarios
RF-02	Buscar vuelos, hoteles y vehículos con filtros de fecha, ciudad y precio	Búsqueda
RF-03	Consultar disponibilidad y precios dinámicos de proveedores	Disponibilidad
RF-04	Crear, modificar y cancelar reservas	Reservas
RF-05	Procesar el pago de reservas y emitir comprobantes	Pagos
RF-06	Coordinar la integración con proveedores externos (aerolíneas, hoteles)	Integraciones
RF-07	Acumular y canjear puntos de fidelización	Fidelización
RF-08	Notificar confirmaciones, cambios y recordatorios de viaje	Notificaciones

4. Requisitos no funcionales (RNF)

Código	Criterio	Requerimiento
RNF-01	Escalabilidad	Soportar picos de búsqueda en temporada alta escalando el buscador de forma independiente
RNF-02	Disponibilidad	La búsqueda y disponibilidad deben operar aunque el servicio de pagos o un proveedor externo falle
RNF-03	Mantenibilidad	Aislar la integración con cada proveedor para actualizarla sin afectar el núcleo de reservas
RNF-04	Seguridad	Cifrado de datos de pago y personales, autenticación y control de acceso por rol
RNF-05	Rendimiento	Resultados de búsqueda de disponibilidad en menos de 3 segundos
RNF-06	Consistencia	Las reservas deben sobrevivir a fallas; el pago debe ser atómico con la confirmación de la reserva

5. Dominios del negocio y microservicios propuestos

Microservicio	Responsabilidad (SRP)	Justificación
Usuarios	Identidad, autenticación y perfil del viajero	Aislado por seguridad de datos personales
Búsqueda	Agregación de disponibilidad y precios de múltiples proveedores	Alto volumen de lecturas en picos; se escala de forma independiente
Reservas	Ciclo de vida de la reserva (creación, modificación, cancelación)	Dominio transaccional central
Pagos	Cobro, reembolsos y comprobantes	Datos sensibles de pago; aislado por seguridad y normativa
Integraciones	Comunicación con aerolíneas, hoteles y arriendo de vehículos	Cada proveedor cambia por separado; Circuit Breaker acota fallas externas
Fidelización	Puntos, niveles y canjes	Reglas de negocio de largo plazo con ciclo propio
Notificaciones	Confirmaciones, cambios y recordatorios	Serverless y reutilizable

6. Arquitectura cloud propuesta

- **Patrones de diseño:** API Gateway, Service Registry, Circuit Breaker (crítico ante proveedores externos lentos), Cola de mensajería (AWS SQS) para eventos de reservas y notificaciones, event-driven para confirmaciones asíncronas.
- **Serverless:** AWS Lambda para consultar proveedores, confirmar reservas en segundo plano, generar comprobantes y enviar notificaciones; API Gateway expone la API.
- **Almacenamiento:** base de datos relacional por dominio (reservas, pagos, fidelización); Amazon S3 para comprobantes y documentos; caché en memoria/distribuida para disponibilidad frecuente de búsquedas.

- **Seguridad:** autenticación JWT, autorización por rol (viajero, operador), cifrado TLS y at-rest, tokenización de tarjetas, registro de auditoría de reservas.
- **Flujos principales:** el viajero busca → Búsqueda consulta proveedores (con Circuit Breaker) → selecciona y crea reserva → evento por cola → Lambda confirma disponibilidad → se procesa el pago → confirmación emitida → notificaciones y fidelización actualizada por eventos.

7. Pauta para el diagrama EP01 (checklist)

☐ Componentes cloud native y microservicios con sus funciones y relaciones (IE4)
☐ Patrones: API Gateway, Service Registry, Circuit Breaker, colas SQS (IE5)
☐ Flujos de comunicación y tecnologías cloud (Lambda, SQS, S3, caché) (IE6)
☐ Puntos de seguridad: autenticación, tokenización de pagos, cifrado, auditoría (IE7)
☐ Flujos de datos críticos: búsqueda → reserva → cola → pago → confirmación → fidelización, con monitoreo (IE8)
☐ Almacenamiento: bases por dominio, S3 para comprobantes, caché de disponibilidad
☐ Criterios de escalabilidad, disponibilidad y mantenibilidad visibles en el diseño